TITAN EYE EYEWEAR E-COMMERCE

DBMS TASK -10 ADVANCED SQL QUERY IMPLEMENTATION

OBJECTIVES:

1. Implement subqueries and nested queries.
2. Find products above average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare an advanced SQL report.

Objective 1: Implement Subqueries and Nested Queries

Display customers who have placed orders.

```
SQL> SELECT *
  2  FROM Customer
  3  WHERE CUSTOMER_ID IN
  4  (
  5      SELECT CUSTOMER_ID
  6      FROM Orders
  7  );

CUSTOMER_ID CUSTOMER_NAME
----------- ----------------------------------------
EMAIL
----------------------------------------------------------
PHONE_NUMBER
---------------
ADDRESS
----------------------------------------------------------
PASSWORD
--------------------------------------------------
        101 Arun Kumar
arun@gmail.com
9876543210

CUSTOMER_ID CUSTOMER_NAME
----------- ----------------------------------------
EMAIL
----------------------------------------------------------
PHONE_NUMBER
```

```
ADDRESS
--------------------------------------------------------
PASSWORD
--------------------------------------------------
Bangalore
arun123


CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------
EMAIL
--------------------------------------------------------
PHONE_NUMBER
---------------
ADDRESS
--------------------------------------------------------
PASSWORD
--------------------------------------------------
        102 Priya Sharma
priya@gmail.com
9876543211

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------
EMAIL
--------------------------------------------------------
PHONE_NUMBER
---------------
ADDRESS
--------------------------------------------------------
PASSWORD
--------------------------------------------------
Chennai
priya123

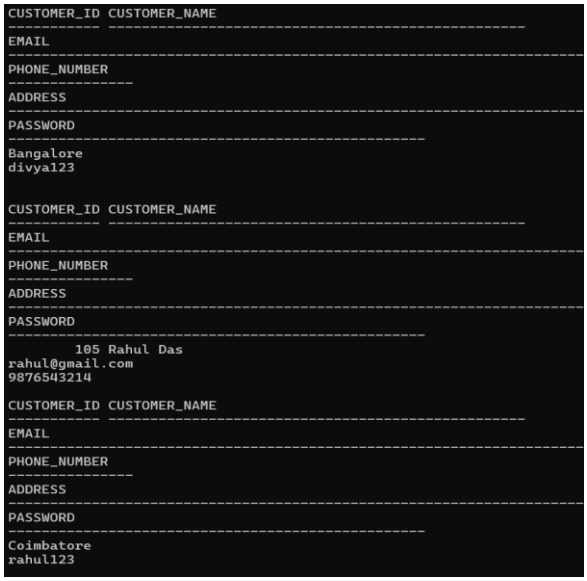

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------
EMAIL
--------------------------------------------------------
```

```
--------------------------------------------------
PHONE_NUMBER
---------------
ADDRESS
--------------------------------------------------
PASSWORD
--------------------------------------------------
        103 Kavin Raj
kavin@gmail.com
9876543212

CUSTOMER_ID CUSTOMER_NAME
----------- ----------------------------------------
EMAIL
--------------------------------------------------
PHONE_NUMBER
---------------
ADDRESS
--------------------------------------------------
PASSWORD
--------------------------------------------------
Chennai
kavin123


CUSTOMER_ID CUSTOMER_NAME
----------- ----------------------------------------
EMAIL
--------------------------------------------------
PHONE_NUMBER
---------------
ADDRESS
--------------------------------------------------
PASSWORD
--------------------------------------------------
        104 Divya S
divya@gmail.com
9876543213

CUSTOMER_ID CUSTOMER_NAME
```



Purpose: To retrieve customer details based on the customer IDs available in the Orders table using a subquery.

Objective 2: Find Products Above Average Price

Display products whose price is above the average product price.

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, PRICE
  2  FROM Product
  3  WHERE PRICE >
  4  (
  5      SELECT AVG(PRICE)
  6      FROM Product
  7  );

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------
     PRICE
----------
       102
Titan Metal Frame
      3299

       105
Titan Premium Frame
      3499

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------
     PRICE
----------
```

Purpose: To identify Titan Eye products whose prices are greater than the average price of all products.

Objective 3: Identify Customers with Maximum Purchases

Find customers with the maximum total purchase amount.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE
  4  FROM Customer C
  5  JOIN Orders O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  HAVING SUM(O.TOTAL_AMOUNT) =
  9  (
 10      SELECT MAX(TOTAL_PURCHASE)
 11      FROM
 12      (
 13          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
 14          FROM Orders
 15          GROUP BY CUSTOMER_ID
 16      )
 17  );

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_PURCHASE
----------- -------------------------------------------------- --------------
        105 Rahul Das                                                    7499
```

Purpose: To identify the customers who have spent the highest total amount on purchases in the Titan Eye database.

Objective 4: Generate Complex Business Queries:

4.1 Display Complete Order Details

Displays customer name, order ID, product name, quantity, price, total amount and order status.

```
SQL> SELECT C.CUSTOMER_NAME,
  2         O.ORDER_ID,
  3         P.PRODUCT_NAME,
  4         OI.QUANTITY,
  5         OI.PRICE,
  6         O.TOTAL_AMOUNT,
  7         O.ORDER_STATUS
  8  FROM Customer C
  9  JOIN Orders O
 10  ON C.CUSTOMER_ID = O.CUSTOMER_ID
 11  JOIN Order_Item OI
 12  ON O.ORDER_ID = OI.ORDER_ID
 13  JOIN Product P
 14  ON OI.PRODUCT_ID = P.PRODUCT_ID;

CUSTOMER_NAME                                      ORDER_ID
-------------------------------------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE TOTAL_AMOUNT ORDER_STATUS
---------- ---------- ------------ ------------------------------
Arun Kumar                                                101
Titan Classic Frame
         2       4500         4500 Shipped

Priya Sharma                                              102
Titan Metal Frame
         1       4800         4999 Shipped

CUSTOMER_NAME                                      ORDER_ID
-------------------------------------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE TOTAL_AMOUNT ORDER_STATUS
---------- ---------- ------------ ------------------------------

Kavin Raj                                                 103
Titan Aviator Sunglasses
         1       5999         5999 Processing
```

```
Divya S                                                    104
Titan Round Sunglasses

CUSTOMER_NAME                                      ORDER_ID
-------------------------------------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE TOTAL_AMOUNT ORDER_STATUS
---------- ---------- ------------ ------------------------------
         1       1999         1999 Delivered

Rahul Das                                                  105
Titan Premium Frame
         1       7499         7499 Pending
```

4.2 Find Best-Selling Products

Displays products according to their total quantity sold, from highest to lowest.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(OI.QUANTITY) AS TOTAL_SOLD
  4  FROM Product P
  5  JOIN Order_Item OI
  6  ON P.PRODUCT_ID = OI.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY TOTAL_SOLD DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
TOTAL_SOLD
----------
       101
Titan Classic Frame
         2

       105
Titan Premium Frame
         1
```

```
       102
Titan Metal Frame

PRODUCT_ID
----------
PRODUCT_NAME
----------------------------------------
TOTAL_SOLD
----------
         1

       103
Titan Aviator Sunglasses
         1
```

4.3 Perform Category-Wise Sales Analysis

Displays total sales for each category, such as Eyeglasses and Sunglasses.

```
SQL> SELECT C.CATEGORY_NAME,
  2         SUM(OI.QUANTITY * OI.PRICE) AS TOTAL_SALES
  3  FROM Category C
  4  JOIN Product P
  5  ON C.CATEGORY_ID = P.CATEGORY_ID
  6  JOIN Order_Item OI
  7  ON P.PRODUCT_ID = OI.PRODUCT_ID
  8  GROUP BY C.CATEGORY_NAME
  9  ORDER BY TOTAL_SALES DESC;

CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
Eyeglasses
      21299

Sunglasses
       7998
```

Objective 5: Prepare an Advanced SQL Report

Generate Customer-Wise Sales Report.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         COUNT(O.ORDER_ID) AS TOTAL_ORDERS,
  4         SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE,
  5         AVG(O.TOTAL_AMOUNT) AS AVERAGE_PURCHASE,
  6         MAX(O.TOTAL_AMOUNT) AS MAX_PURCHASE,
  7         MIN(O.TOTAL_AMOUNT) AS MIN_PURCHASE
  8  FROM Customer C
  9  JOIN Orders O
 10  ON C.CUSTOMER_ID = O.CUSTOMER_ID
 11  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
 12  ORDER BY TOTAL_PURCHASE DESC;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
TOTAL_PURCHASE AVERAGE_PURCHASE MAX_PURCHASE MIN_PURCHASE
-------------- ---------------- ------------ ------------
        105 Rahul Das                                                     1
          7499             7499         7499         7499

        103 Kavin Raj                                                     1
          5999             5999         5999         5999

        102 Priya Sharma                                                  1
          4999             4999         4999         4999


CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
TOTAL_PURCHASE AVERAGE_PURCHASE MAX_PURCHASE MIN_PURCHASE
-------------- ---------------- ------------ ------------
        101 Arun Kumar                                                    1
          4500             4500         4500         4500

        104 Divya S                                                       1
          1999             1999         1999         1999
```

Purpose: To generate a detailed customer-wise sales report showing total orders, total purchase amount, average purchase amount, maximum purchase and minimum purchase for each customer.